

CoCotb-Based Verification Environment

For the 8-bit Combinational ALU

Presented to:

Analog Devices

Eng. Mohamed El Adawy

Presented by:

Mohamed Abdelhafez Elnewehy

July 2026

Verification Environment Report

1. Overview

This document explains the cocotb-based verification environment built for the 8-bit combinational ALU. It follows the same layered testbench philosophy as a class-based (SystemVerilog/UVM-style) environment — a generator creates stimulus, a driver applies it to the DUT, a monitor observes the DUT's response, and a scoreboard checks that response against a reference model — but implements every piece as a plain Python object driven by cocotb coroutines instead of SystemVerilog classes. All components are grouped inside an Environment, which is instantiated and run directly from the cocotb test function.

2. Architecture

The diagram below shows how the Python files map onto the standard testbench hierarchy, using the exact class and file names used in the code.



Figure 1 Simplified diagram (actual one is in class-based report)

3. Component Descriptions

design.sv — *simple_alu* (DUT)

A purely combinational SystemVerilog module. It takes 8-bit operands A and B and a 3-bit opcode, and produces Result, Zero, Carry and Overflow. Six opcodes are implemented (ADD, SUB, AND, OR, SHL, SHR); the remaining two 3-bit encodings are invalid and force Result to zero. A 9-bit internal accumulator captures the carry-out on ADD and the borrow on SUB, and Overflow is derived from the sign bits of the operands and the result, per the standard two's-complement overflow rule.

transaction.py

Defines the plain data object that moves through the environment. It holds the stimulus fields (A, B, opcode) and the response fields (Result, Zero, Carry, Overflow) that get filled in once the DUT has reacted. Unlike a SystemVerilog transaction class, there is no built-in `randomize()` or `copy()` — field values are assigned directly by whichever component owns the object at that stage of the pipeline.

generator.py

Produces the stream of transactions for one test phase. Given a transaction count and a list of legal opcodes for that phase, it creates a fresh Transaction each iteration, fills A and B with random 8-bit values (`random.randint(0, 255)`), picks an opcode via `random.choice()` from the supplied list, and pushes the transaction into the `gen2drv` queue with `await gen2drv.put(tx)`.

driver.py

Continuously pulls transactions out of `gen2drv` and drives A, B and opcode onto the DUT pins (`self.dut.A.value = tx.A`, etc.). Because the ALU is combinational, the driver simply waits 5 ns with `Timer(5, units="ns")` for the logic to settle before looping back to fetch the next transaction — there is no explicit interface or clock-edge synchronization as there would be with a SystemVerilog virtual interface.

monitor.py

Runs in lock-step with the driver, waiting the same 5 ns before sampling. On each pass it reads A, B, opcode, Result, Zero, Carry and Overflow directly off the dut handle, packages them into a new Transaction, and forwards it to the scoreboard through the `mon2sb` queue. The monitor never drives a signal — it only observes.

scoreboard.py

Pulls each observed transaction from `mon2sb` and checks it against a golden reference model re-implemented independently in Python inside `check_output()`. For ADD/SUB it reproduces the 9-bit accumulator arithmetic to derive the expected Result, Carry and Overflow; for AND/OR/SHL/SHR it computes the expected Result directly; Zero is derived from the expected Result in every case. Every transaction is logged as PASS or FAIL, with both a per-category counter (`cat_passed` / `cat_failed`) and a running total (`total_passed` / `total_failed`) that feeds the final summary.

agent.py

A lightweight container that owns one Generator, one Driver and one Monitor, and wires the `gen2drv` queue between the generator and the driver. `start_tasks()` launches the driver and monitor as free-running background coroutines with `cocotb.start_soon()`; `run_phase()` drives the generator through exactly one batch of transactions for the current test phase.

environment.py

The top of the verification hierarchy. It creates the `mon2sb` queue, builds the Agent and the Scoreboard around it, and orchestrates three sequential test phases — Arithmetic (opcodes 0–1), Logic (opcodes 2–3) and Shift (opcodes 4–5) — each running 100 transactions. For every phase it starts a scoreboard task sized to that phase's transaction count, runs the generator for that phase, awaits the scoreboard task, and inserts a 10 ns gap before moving to the next phase. It finishes by printing the final combined pass/fail summary.

testbench.py

The `cocotb` test entry point, decorated with `cocotb.test()`. It simply instantiates the Environment with the dut handle that `cocotb` provides and awaits `env.run()`; all sequencing detail is delegated to the Environment.

4. How a Transaction Flows Through the Environment

- The Generator creates a transaction with random A, B and an opcode chosen from the current phase's list, and puts it into gen2drv.
- The Driver takes it from gen2drv and drives A, B and opcode onto the DUT pins.
- The Driver waits 5 ns for the combinational logic to settle.
- The Monitor, waiting the same 5 ns, samples the DUT's inputs and outputs and forwards a transaction to mon2sb.
- The Scoreboard takes the transaction from mon2sb, computes the expected values with its own reference model, and logs PASS or FAIL.
- Once every transaction in the phase has been generated and checked, the Environment prints that phase's summary and moves on; after all three phases it prints the final combined total.

5. Stimulus Strategy and Coverage Considerations

Stimulus is organized into three phases so that each opcode family is exercised in isolation and reported separately: Arithmetic (ADD/SUB), Logic (AND/OR) and Shift (SHL/SHR), 100 transactions each, for 300 transactions in total. Within a phase, both operands and the opcode choice are fully randomized — there are no directed, hand-picked corner-case transactions of the kind a class-based SystemVerilog environment typically injects first (e.g. a fixed A=255, B=1 to force carry-out).

With 100 random samples per phase the classic boundary conditions — carry-out and borrow, signed overflow in both directions, the zero flag, and shift amounts at 0 and 7 — are likely to appear at least once by chance, and the results below confirm several of them did occur, but this is a probabilistic guarantee, not a deterministic one. A directed pre-phase (mirroring the 14 directed iterations used in the class-based environment) would give the verification plan a guaranteed hit on every named corner case regardless of the random seed, and is the most valuable next enhancement to this environment.

6. Results Snippet

Simulator: Icarus Verilog on EDA playground, run through cocotb via make. Actual console output from a run of this environment:

```
--- STARTING ARITHMETIC TESTS ---
Opcode: 000 | A: 171 | B: 003 | Res: 174 (Exp: 174) | Status: PASS
Opcode: 000 | A: 224 | B: 175 | Res: 143 (Exp: 143) | Status: PASS
Opcode: 001 | A: 049 | B: 252 | Res: 053 (Exp: 053) | Status: PASS
Opcode: 000 | A: 238 | B: 255 | Res: 237 (Exp: 237) | Status: PASS
Opcode: 001 | A: 224 | B: 168 | Res: 056 (Exp: 056) | Status: PASS
...

=====
[ARITHMETIC (ADD/SUB) SUMMARY] PASSED: 100 | FAILED: 0
=====

--- STARTING LOGIC TESTS ---
Opcode: 001 | A: 244 | B: 119 | Res: 125 (Exp: 125) | Status: PASS
Opcode: 001 | A: 244 | B: 119 | Res: 125 (Exp: 125) | Status: PASS
Opcode: 011 | A: 127 | B: 058 | Res: 127 (Exp: 127) | Status: PASS
Opcode: 010 | A: 132 | B: 209 | Res: 128 (Exp: 128) | Status: PASS
Opcode: 010 | A: 101 | B: 088 | Res: 064 (Exp: 064) | Status: PASS
...

=====
[LOGIC (AND/OR) SUMMARY] PASSED: 100 | FAILED: 0
=====

--- STARTING SHIFT TESTS ---
Opcode: 011 | A: 254 | B: 068 | Res: 254 (Exp: 254) | Status: PASS
Opcode: 010 | A: 006 | B: 184 | Res: 000 (Exp: 000) | Status: PASS
Opcode: 100 | A: 087 | B: 127 | Res: 128 (Exp: 128) | Status: PASS
Opcode: 101 | A: 041 | B: 139 | Res: 005 (Exp: 005) | Status: PASS
Opcode: 101 | A: 095 | B: 160 | Res: 095 (Exp: 095) | Status: PASS
...

=====
[SHIFT (SHL/SHR) SUMMARY] PASSED: 100 | FAILED: 0
=====

*****
*** FINAL TOTAL SUMMARY ***
*** TOTAL PASSED: 300 ***
*** TOTAL FAILED: 0 ***
*****
```

All 300 transactions across the three phases passed, with the scoreboard's independent reference model agreeing with the DUT on Result, Zero, Carry and Overflow in every case.